COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE10 Data Structures and Algorithm	**SECTION:**	CPE21S4
DATE PERFORMED:	9/15/26	**DATE SUBMITTED:**	9/15/26
NAME:	Peñaflorida,Wrio Carlo	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HOA.8.1 **SORTING ALGORITHMS: SHELL, MERGE, AND QUICK SOR**

1. PROCEDURE OUTPUT

ILO A: Create C++ code for implementation of shell, merge and quick sort.
Preparation Task:
Create an array of elements with random values.
The array must contain 100 elements that are not sorted.
Use the created array for each sorting algorithm below. Show the output of the preparation task on table 8-1 in section 6.
Create a header file for the implementation of the different sorting algorithms.
Import this header file into your main.cpp.

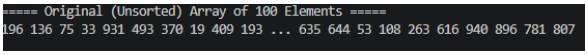
Code + Console Screenshot	
Observations	

Table 8-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	
Observations	

Table 8-2. Shell Sort Technique

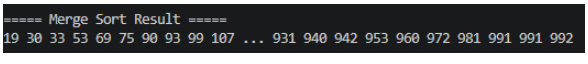
Code + Console Screenshot	
Observations	

Table 8-3. Merge Sort Algorithm

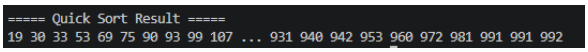
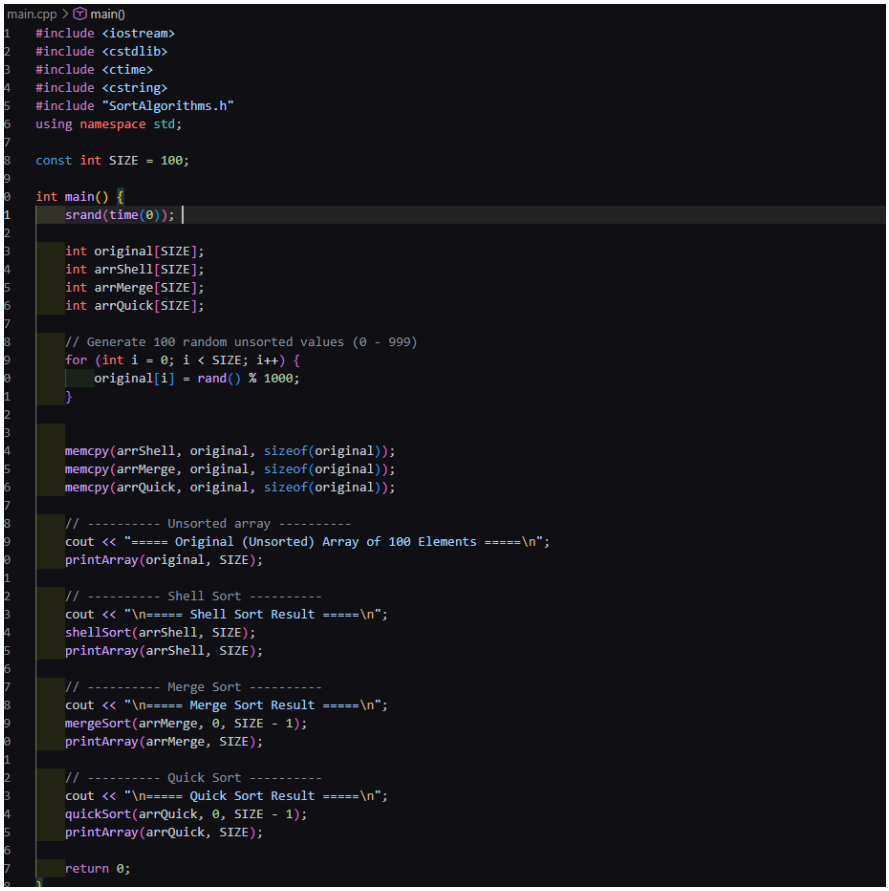
Code + Console Screenshot	
Observations	

Table 8-4. Quick Sort Algorithm

Main.cpp



```cpp
main.cpp > main()
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <cstring>
#include "SortAlgorithms.h"
using namespace std;

const int SIZE = 100;

int main() {
    srand(time(0));

    int original[SIZE];
    int arrShell[SIZE];
    int arrMerge[SIZE];
    int arrQuick[SIZE];

    // Generate 100 random unsorted values (0 - 999)
    for (int i = 0; i < SIZE; i++) {
        original[i] = rand() % 1000;
    }


    memcpy(arrShell, original, sizeof(original));
    memcpy(arrMerge, original, sizeof(original));
    memcpy(arrQuick, original, sizeof(original));

    // ---------- Unsorted array ----------
    cout << "===== Original (Unsorted) Array of 100 Elements =====\n";
    printArray(original, SIZE);

    // ---------- Shell Sort ----------
    cout << "\n===== Shell Sort Result =====\n";
    shellSort(arrShell, SIZE);
    printArray(arrShell, SIZE);

    // ---------- Merge Sort ----------
    cout << "\n===== Merge Sort Result =====\n";
    mergeSort(arrMerge, 0, SIZE - 1);
    printArray(arrMerge, SIZE);

    // ---------- Quick Sort ----------
    cout << "\n===== Quick Sort Result =====\n";
    quickSort(arrQuick, 0, SIZE - 1);
    printArray(arrQuick, SIZE);

    return 0;
}
```

Sort.h

SortAlgorithms.h > merge(int [], int, int, int)

```cpp
#ifndef SORTALGORITHMS_H
#define SORTALGORITHMS_H

#include <iostream>
using namespace std;

// ============================================================
// A.1 SHELL SORT
// ============================================================
void shellSort(int arr[], int n) {
    for (int gap = n / 2; gap > 0; gap /= 2) {
        for (int i = gap; i < n; i++) {
            int temp = arr[i];
            int j = i;
            while (j >= gap && arr[j - gap] > temp) {
                arr[j] = arr[j - gap];
                j -= gap;
            }
            arr[j] = temp;
        }
    }
}

// ============================================================
// A.2 MERGE SORT
// ============================================================
void merge(int arr[], int left, int middle, int right) {
    int n1 = middle - left + 1;
    int n2 = right - middle;

    int* leftArr = new int[n1];
    int* rightArr = new int[n2];

    for (int i = 0; i < n1; i++)
        leftArr[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        rightArr[j] = arr[middle + 1 + j];

    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2) {
        if (leftArr[i] <= rightArr[j])
            arr[k++] = leftArr[i++];
        else
            arr[k++] = rightArr[j++];
    }
    while (i < n1) arr[k++] = leftArr[i++];
    while (j < n2) arr[k++] = rightArr[j++];

    delete[] leftArr;
    delete[] rightArr;
}

void mergeSort(int arr[], int left, int right) {
    if (left >= right) return; // 0 or 1 element = already sorted

    int middle = left + (right - left) / 2;
```

```cpp
void mergeSort(int arr[], int left, int right) {
    if (left >= right) return; // 0 or 1 element = already sorted

    int middle = left + (right - left) / 2;
    mergeSort(arr, left, middle);       // sort first half
    mergeSort(arr, middle + 1, right);  // sort second half
    merge(arr, left, middle, right);    // merge sorted halves
}

// ============================================================
// A.3 QUICK SORT
// ============================================================
int partition(int arr[], int low, int high) {
    int pivot = arr[high]; // last element as pivot
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1; // pivot's final sorted position
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pivotIndex = partition(arr, low, high);
        quickSort(arr, low, pivotIndex - 1);   // sort left sub-list
        quickSort(arr, pivotIndex + 1, high);  // sort right sub-list
    }
}

// ============================================================
// UTILITY: print array (only first/last few if large)
// ============================================================
void printArray(int arr[], int n) {
    int limit = (n > 20) ? 10 : n;
    for (int i = 0; i < limit; i++)
        cout << arr[i] << " ";
    if (n > 20) {
        cout << "... ";
        for (int i = n - 10; i < n; i++)
            cout << arr[i] << " ";
    }
    cout << endl;
}

#endif
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

Problem 1.

Can we sort the left/right sub-lists from quicksort's partition using other sorting algorithms? Demonstrate an example.

Yes. Quicksort's only requirement after partition() is that both resulting sub-lists end up sorted — *how* they get sorted doesn't matter. Recursive quicksort is just the "default" choice, not a requirement.

This is actually done in real-world libraries as an optimization called a hybrid sort. A common pattern: once a sub-list shrinks below a small threshold (e.g., 10 elements), switch to Insertion Sort instead of recursing further with quicksort, since Insertion Sort has less overhead on tiny arrays.

Example: Using the array {10, 42, 15, 12, 37, 7, 9} from Image 4 — after the first partition (pivot = 9), you get sub-lists {7} and {10, 42, 15, 12, 37}. Instead of recursively calling quickSort() on {10, 42, 15, 12, 37}, you could call insertionSort({10, 42, 15, 12, 37}) directly since it's small, and it would sort it to {10, 12, 15, 37, 42} just as correctly. Real-world "introsort" implementations (used in C++'s std::sort) do exactly this.

Problem 2: For {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74} **— which sort is fastest, and why do Merge Sort and Quick Sort both achieve O(N log N)?**

For an array this small (15 elements) and randomly arranged, Quick Sort typically gives the fastest *practical* performance. Here's why, and how it compares:

- Quick Sort partitions in-place — no extra array is allocated. Since this data is random (not sorted or reverse-sorted), the pivot choices tend to split the array reasonably evenly, so it hits its average-case O(n log n) with low constant overhead and good cache locality.
- Merge Sort also runs in O(n log n), but it needs extra memory (new int[n1], new int[n2] in the code above) for every merge step. For small arrays, that allocation overhead can make it slightly slower in wall-clock time than Quick Sort, even though both share the same Big-O class.
- In practice, for an array this tiny, the difference is milliseconds and barely measurable — but Quick Sort generally "wins" in real benchmarks on random data of this size.

Why both reach O(N log N):

- Merge Sort *always* splits the array exactly in half regardless of the data values (middle = left + (right-left)/2), so it always takes log n levels of recursion, and each level does O(n) work to merge — giving a guaranteed O(n log n) in best, worst, *and* average cases.
- Quick Sort reaches O(n log n) only when the pivot happens to split the array into two roughly balanced halves each time — that gives log n recursion depth with O(n) partitioning work per level, same shape as Merge Sort. This is why Quick Sort's O(n log n) is an *average/best* case, not guaranteed — a poor pivot (e.g., always picking the smallest/largest element, which happens on already-sorted input) creates lopsided partitions and degrades to $O(n^2)$.

Conclusion

This activity demonstrated three different approaches to sorting: Shell Sort, which improves on Insertion Sort by comparing elements at decreasing gaps before a final cleanup pass; Merge Sort, which guarantees O(n log n) performance by always splitting the array in half regardless of the data; and Quick Sort, which partitions in-place around a pivot for fast average-case performance but risks $O(n^2)$ on poorly chosen pivots. Implementing all three in C++ and testing them on the same 100-element array confirmed that each algorithm reaches the same correct sorted result through very different processes. The supplementary activity showed that Quick Sort's partitioning step can be paired with other sorting methods for small sub-lists, and that algorithms sharing the same Big-O complexity can still differ in real-world speed due to memory overhead. Overall, this exercise reinforced how algorithm choice depends on the data's size and arrangement, not just its theoretical time complexity. Areas I'd like to review further include tracing recursion depth in Merge Sort and understanding pivot selection strategies that avoid Quick Sort's worst case.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.